Tinpro01-7 Practicumopdracht 3: encryptie

Wessel Oele

9 februari 2025

Lees eerst de hele opdracht alvorens te beginnen.

Inleiding

In deze opdracht gaan we ons eigen encryptiealgoritme programmeren. Ons algoritme vertoont enige gelijkenis met de manier waarop het bekende AES encryptie algoritme werkt. Ons algoritme:

- is tekst gebaseerd.
- is niet precies gelijk aan AES, maar komt daar wel bij in de buurt.
- is, net als AES, gebaseerd op een zogeheten *sp-netwerk*: een *substitution permutation network*. Veel bekende encryptiealgoritmen zijn sp-netwerken. Voorbeelden: Aes, Des, Triple Des, Rijndael, etc.
- is een zogeheten *blockcypher*: Een gegeven tekst wordt opgesplitst in blokken van een vaste grootte. Hierover later meer.

We gaan in deze opdracht een aantal kleine, relatief eenvoudige, functies schrijven. Deze functies worden gebruikt in weer ingewikkeldere functies, enzovoorts. De gedachte hierachter is een zeer belangrijke:

- We schrijven eenvoudige kleine functies die eenvoudige taken voor ons uitvoeren.
- Eenvoudige functies worden gecombineerd in functies die complexere taken uitvoeren. Deze complexere functies blijven echter relatief klein.
- Complexere functies combineren we in nog complexere (maar nog steeds kleine) functies, enzovoorts...

Het opbouwen van complexiteit uit eenvoudige bouwstenen zorgt ervoor dat:

- we voortdurend begrijpen wat we aan het doen zijn.
- een complex programma begrijpelijk blijft daar we de complexiteit opbouwen uit eenvoudigere bouwstenen.
- een programma modulair en onderhoudbaar blijft als we het bovenstaande netjes en gestructureerd toepassen.

Opdracht 1: Libraries en teststrings

- Maak de module `Encryption` en importeer de libraries `Data.Bits, Data.Char, Data.List`.
- Zet in je source 3 teststrings en noem deze `t1,t2,t3`: Een kleine van hooguit 15 letters, een grotere en een nog grotere van een paar zinnen. Het direct in de sourcecode plaatsen van strings is uiteraard niet fraai, maar het is wel een manier om makkelijk en snel de functies die we gaan schrijven te testen op deze "input".
- Plaats de variabele `testkey` in je source. Dit is een `String` met een lengte van 16 characters.

Opdracht 2: basisfuncties

Schrijf de functies:

- `String2Ints :: String -> [Int]`
  Deze functie maakt van een `String` een lijst van `Int`. Tip: Bestudeer de `Data.Char` library.
- `Ints2String :: [Int] -> String`
  Deze functie is de *inverse* van `String2Ints` en doet het tegenovergestelde: Een lijst van `Int` wordt omgezet in een `String`.

Padding

Zoals eerder beschreven zijn we bezig een blockcypher te programmeren: Een gegeven tekst zal moeten worden opgesplitst in een lijst van blokken met een vaste grootte. In ons algoritme kiezen we voor een blokgrootte van 16 characters. Er ontstaat nu een probleem: Een `String` die moet worden opgesplitst zal doorgaans niet een lengte hebben die een veelvoud is van 16. We lossen

dit op door de `String` op te vullen met extra dummie characters zodat de totale lengte een veelvoud van 16 is. Het Engelse werkwoord *to pad* betekent opvullen. . . Schrijf de volgende functies:

- `pad :: Int -> [Int] -> [Int]`: Deze functie zal aan een lijst getallen een aantal extra getallen toevoegen, zodat de de lengte van de lijst een veelvoud is van een bepaalde *n*. Voorbeeld: `pad 5 [1,2,3,4,5,6,7]` levert: `[1,2,3,4,5,6,7,0,0,0]`
- `hack :: Int -> [a] -> [[a]]`: Deze functie hakt een lijst van elementen in stukken met een lengte *l*.
  Voorbeeld: `hack 4 [1,2,3,4,5,6,7,8,9,10,11,12]`
  levert [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Blocks

Plaats onder je eerder gemaakte teststrings de volgende typesynoniemen:

- `type Block = [[Int]]`
- `type Key = [Int]`

We kunnen deze typesoniemen gebruiken in de typesignatures van onze functies. Deze worden daardoor leesbaarder met als gevolg schonere en begrijpelijkere code. Het is belangrijk goed te begrijpen wat we onder een `Block` verstaan. Eén `Block` is op te vatten als een matrix:

$$abcdefghijklmnop = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix}$$

In Haskell coderen we één `Block` als een lijst van lijsten. Voorbeeld:

```
abcdefghijklmnop = [[a,b,c,d],[e,f,g,h],[i,j,k,l],[m,n,o,p]]
```

Schrijf de functies:

- `string2blocks::String -> [Block]`
  Deze functie:
  - converteert alle `Char` in de `String` naar ASCII waarden.
  - padt een lijst op een veelvoud van 16.
  - maakt van die lijst een lijst van blokken.
- `blocks2string :: [Block] -> String`
  Deze functie is de inverse van `String2blocks`.

Opdracht 3: Encryptiefuncties: permutatie

Schrijf de volgende functies:

- `rotate :: Int -> [a] -> [a]`
  Deze functie roteert een lijst `n` posities zoals uitgelegd in de les. `n` kan ook negatief zijn, kortom:
  `[1,2,3,4] = rotate (-2) (rotate 2 [1,2,3,4]) = [1,2,3,4]`
  Je mag bij het schrijven van deze functie aannemen dat een lijst altijd een lengte van 4 heeft.

- `rotblock :: Block -> Block`
  Deze functie roteert elk van de 4 rijen in een `Block` op de volgende wijze:

  - rij 1: geen rotatie
  - rij 2: roteer 1 positie
  - rij 3: roteer 2 posities
  - rij 4: roteer 3 posities

- `invrotblock :: Block -> Block`
  De inverse van `rotblock`

- `rotblockbycolumn :: Block -> Block`
  Doet hetzelfde als `rotblock` maar nu over de *kolommen*.

- `invrotblockbycolumn :: Block -> Block`
  De inverse van `invrotblockbycolumn`.

Opdracht 4: Encryptiefuncties: sleutelgeneratie en het Vernam cypher

Voor het bouwen van een sp-netwerk zullen we uit één sleutel een aantal rondesleutels moeten berekenen. We houden het hierbij eenvoudig: Uit één sleutel maken we 10 rondesleutels door een gegeven sleutel te roteren. Schrijf de volgende functies:

- `rotatelist :: [a] -> Int -> [a]`
  Deze functie roteert een lijst *n* posities. De lijst kan iedere willekeurige lengte hebben.

- `genroundkeys :: Key -> Int -> [Key]`
  Deze functie genereert uit één sleutel een lijst van sleutels. Het aantal sleutels is even groot als de `Int` waarde. Iedere sleutel in de te genereren lijst is één positie verder geroteerd.
- `block2list :: Block -> [Int]`
  Deze functie maakt van één `Block` een `[Int]`
- `blockxorkey :: Block -> Key -> Block`
  Deze functie voert Vernam encryptie uit op één `Block` en één `Key`.

Opdracht 5: Encryptiefuncties: substitutie

We gebruiken in ons algoritme het Vigenère cypher zoals uitgelegd in de les. Schrijf de volgende functies:

- `vigenere :: [Int] -> [Int] -> [Int]`
  Voert een Vigenère cypher uit door bij elk element uit de eerste lijst het element uit de tweede lijst op te tellen.
- `invvigenere :: [Int] -> [Int] -> [Int]`
  De inverse van de `vigenere` functie.
- `vigenereblock :: Block -> Key -> Block`
  Deze functie voert een Vigenère cypher uit op een `Block` en een `Key`.
- `invvigenereblock :: Block -> Key -> Block`
  De inverse van `vigenereblock`.

Opdracht 6: Encryptiefuncties

In de voorgaande opdrachten heb je alles klaar gezet om een gegeven tekst met een sleutel te kunnen versleutelen. We kunnen nu:

- Tekst naar een lijst getallen (en vervolgens blokken) omzetten en omgekeerd.
- Uit één sleutel een lijst van sleutels genereren.
- Substitutie, permutatie en de $\oplus$ bewerking uitvoeren op blokken.

Hoog tijd om wat we hebben te combineren. . . Schrijf de volgende functies:

- `encryptround :: Block -> Key -> Block`
  Deze functie:
  - roteert de rijen van een `Block`.
  - roteert van het verkregen `Block` vervolgens de kolommen.
  - voert vervolgens een Vigenère versleuteling uit op het verkregen `Block` met de `Key`.
  - voert vervolgens Vernam versleuteling uit op het verkregen `Block` met de `Key`.
  - Geeft het versleutelde `Block` terug.
- `decryptround :: Block -> Key -> Block`
  De inverse van `encryptround`.
- `encryptrounds :: Key -> Block -> Block`
  Deze functie:
  - genereert 10 ronde sleutels uit de sleutel.
  - voert de `encryptround` 10 keer achter elkaar uit op het gegeven `Block`.

  Belangrijk:
  - In elke ronde wordt het resultaat van de voorgaande ronde als "input" gebruikt.
  - in elke ronde wordt uiteraard één van de gegeneerde rondesleutels gebruikt.
- `decryptrounds :: Key -> Block -> Block`
  De inverse van `encryptrounds`.
- `encrypt :: String -> String -> String`
  Deze functie neemt een boodschap en een sleutel en geeft de versleutelde boodschap terug. Werking:
  - De boodschap wordt geconverteerd naar een lijst van blokken.
  - Elk `Block` wordt versleuteld m.b.v. de `Encryptrounds` functie en een uit de sleutel verkegen rondesleutel.
  - De versleutelde blokken worden geconverteerd naar een String.
- `decrypt :: String -> String -> String`
  De inverse van `Encrypt`.